Sprawozdanie z projektu – **Szyfr Mendelejewa (2020)**

Autor: Nikodem Płowucha

Programowanie I, grupa 1, Wydział Matematyki Stosowanej, Politechnika Śląska

1. Temat projektu

Projekt polega na implementacji szyfru Mendelejewa – algorytmu, który zamienia litery tekstu na numery atomowe pierwiastków chemicznych odpowiadających ich symbolom. Symbole pierwiastków mogą być jedno- lub dwuliterowe.

```c
// Definicja struktury przechowującej symbol i numer atomowy
typedef struct {
    const char* symbol;
    int number;
} Element;

// Słownik 118 pierwiastków chemicznych
Element elements[] = {
    {"h",1},{"he",2},{"li",3},{"be",4},{"b",5},{"c",6},{"n",7},{"o",8},{"f",9},{"ne",10},
    {"na",11},{"mg",12},{"al",13},{"si",14},{"p",15},{"s",16},{"cl",17},{"ar",18},{"k",19},{"ca",20},
    {"sc",21},{"ti",22},{"v",23},{"cr",24},{"mn",25},{"fe",26},{"co",27},{"ni",28},{"cu",29},{"zn",30},
    {"ga",31},{"ge",32},{"as",33},{"se",34},{"br",35},{"kr",36},{"rb",37},{"sr",38},{"y",39},{"zr",40},
    {"nb",41},{"mo",42},{"tc",43},{"ru",44},{"rh",45},{"pd",46},{"ag",47},{"cd",48},{"in",49},{"sn",50},
    {"sb",51},{"te",52},{"i",53},{"xe",54},{"cs",55},{"ba",56},{"la",57},{"ce",58},{"pr",59},{"nd",60},
    {"pm",61},{"sm",62},{"eu",63},{"gd",64},{"tb",65},{"dy",66},{"ho",67},{"er",68},{"tm",69},{"yb",70},
    {"lu",71},{"hf",72},{"ta",73},{"w",74},{"re",75},{"os",76},{"ir",77},{"pt",78},{"au",79},{"hg",80},
    {"tl",81},{"pb",82},{"bi",83},{"po",84},{"at",85},{"rn",86},{"fr",87},{"ra",88},{"ac",89},{"th",90},
    {"pa",91},{"u",92},{"np",93},{"pu",94},{"am",95},{"cm",96},{"bk",97},{"cf",98},{"es",99},{"fm",100},
    {"md",101},{"no",102},{"lr",103},{"rf",104},{"db",105},{"sg",106},{"bh",107},{"hs",108},{"mt",109},
    {"ds",110},{"rg",111},{"cn",112},{"nh",113},{"fl",114},{"mc",115},{"lv",116},{"ts",117},{"og",118}
};

int element_count = sizeof(elements) / sizeof(Element);

// Użyta funkcja: find_element() – szuka symbolu pierwiastka w tablicy
int find_element(const char* sym) {
    for (int i = 0; i < element_count; i++) {
        if (strcmp(elements[i].symbol, sym) == 0)
            return elements[i].number;
    }
    return -1;
}
```

2. Opis pobieranych danych

Program pobiera od użytkownika ciąg znaków (tekst złożony z liter A–Z). Dane wejściowe mogą zawierać wiele słów rozdzielonych spacją.

Fragmenty z funkcji main

```
    // --- Walidacja Startowa (Menu Wyboru) ---
    while (1) {
        printf("\n### SZYFROWANIE SYMBOLAMI PIERWIASTKOW ###\n");
        printf("1. Rozpocznij szyfrowanie\n");
        printf("2. Wyjdz z programu\n");
        printf("Wybierz opcje (1 lub 2): ");

        if (scanf("%d", &choice) != 1) {
            while (getchar() != '\n');
            printf("Nieprawidlowy format wejscia. Sprobuj ponownie.\n");
            continue;
        }

        while (getchar() != '\n');

        if (choice == 1) {
            break;
        }
        else if (choice == 2) {
            printf("Program zostal zakonczony.\n");
            exit(0);
        }
        else {
            printf("Nieprawidlowy wybor. Wpisz 1 lub 2.\n");
        }
    }
    // --- Koniec Walidacji Startowej ---
```

```
// Pobieranie tekstu
printf("\n--- Rozpoczeto Szyfrowanie ---\n");
printf("Podaj tekst do zaszyfrowania: ");
// Wczytywanie wejścia
if (fgets(text, sizeof(text), stdin) == NULL) {
    text[0] = '\0'; // Zabezpieczenie na wypadek błędu wczytywania
}
else {
    text[strcspn(text, "\n")] = 0;
}

// Walidacja znaków wejściowych
for (i = 0; text[i] != '\0'; i++) {
    if (!isalpha(text[i]) && !isspace(text[i])) {
        printf("Blad walidacji danych: Tekst moze zawierac tylko litery i spacje. Znaleziono niedozwolony znak: '%c'.\n", text[i]);
        return 1;
    }
}
```

3. Opis otrzymywanych rezultatów

Program zwraca tekst zaszyfrowany, gdzie każda litera lub para liter zostaje zastąpiona liczbą atomową pierwiastka. Gwiazdka (*) oddziela liczby w ramach jednego słowa, a podwójna gwiazdka (**) oddziela kolejne słowa.

```
    // Kopiowanie tekstu (strtok modyfikuje oryginalny ciąg)
    strcpy(temp_text, text);
    // Inicjalizacja tokenizacji
    word = strtok(temp_text, " ");

    // Główna pętla szyfrowania słów
    while (word != NULL) {
        // Resetowanie licznika dla nowego słowa
        out_len = 0;
        if (strlen(word) == 0) {
            word = strtok(NULL, " ");
            continue;
        }
        if (!encode_word(word, 0, strlen(word), output, &out_len)) {
            success = 0;
            break;
        }
        if (!first_word) printf("**");
        for (i = 0; i < out_len; i++) {
            printf("%d", output[i]);
            if (i < out_len - 1) printf("*");
        }

        first_word = 0;
        word = strtok(NULL, " ");
    }
    if (!success) {
        printf("\nNie da sie zaszyfrowac podanego tekstu tym sposobem.\n");
    }
    else {
        printf("\n");
    }
    return 0;
}
```

4. Opis algorytmu

Algorytm działa rekurencyjnie. Dla każdej pozycji w słowie:

- Próbuje dopasować symbol dwuliterowy.

- Jeśli nie pasuje – próbuje symbol jednoliterowy.

- Jeśli dopasowanie jest możliwe – dodaje liczbę atomową do bufora.

- Jeśli nie – wraca (backtracking).

Jeśli rekurencja dojdzie do końca słowa – słowo jest poprawnie szyfrowalne.

```c
// Rekurencyjna funkcja próbująca zakodować słowo
int encode_word(const char* word, int pos, int len, int output[], int* out_len) {
    if (pos == len) return 1; // sukces
    char sym[3] = { 0 };

    // spróbuj 2-literowy symbol
    if (pos + 2 <= len) {
        sym[0] = tolower(word[pos]);
        sym[1] = tolower(word[pos + 1]);
        int num = find_element(sym);
        if (num != -1) {
            output[(*out_len)++] = num;
            if (encode_word(word, pos + 2, len, output, out_len))
                return 1;
            (*out_len)--; // cofamy (backtracking)
        }
    }

    // spróbuj 1-literowy symbol
    sym[0] = tolower(word[pos]);
    sym[1] = '\0';
    int num = find_element(sym);
    if (num != -1) {
        output[(*out_len)++] = num;
        if (encode_word(word, pos + 1, len, output, out_len))
            return 1;
        (*out_len)--; // cofamy (backtracking)
    }
    return 0;
}
```

Schemat blokowy ASCII (ogólny):

START

|

V

Pobierz tekst

|

Podziel na słowa

|

Dla każdego słowa:

|

V

Rekurencyjne dopasowanie symboli

|

+-- NIE -> komunikat o błędzie

|

+-- TAK -> dopisz wynik

|

Połącz słowa **

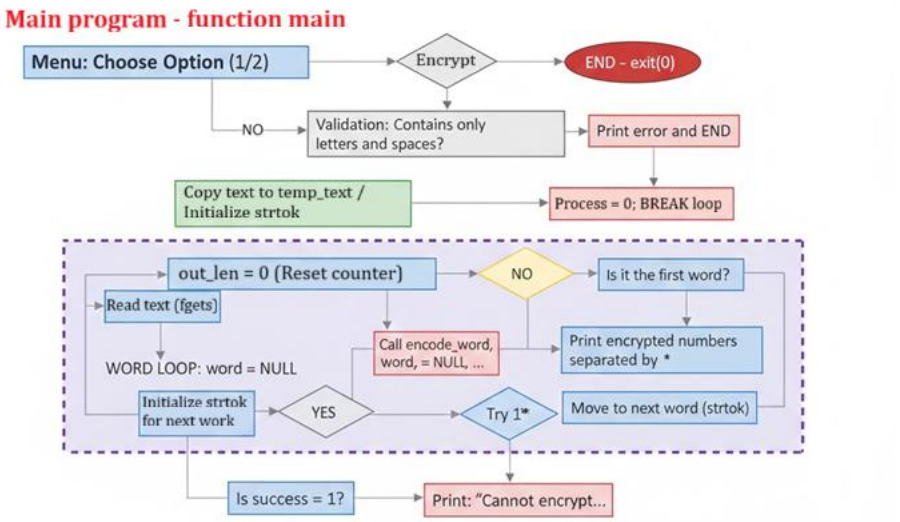
|

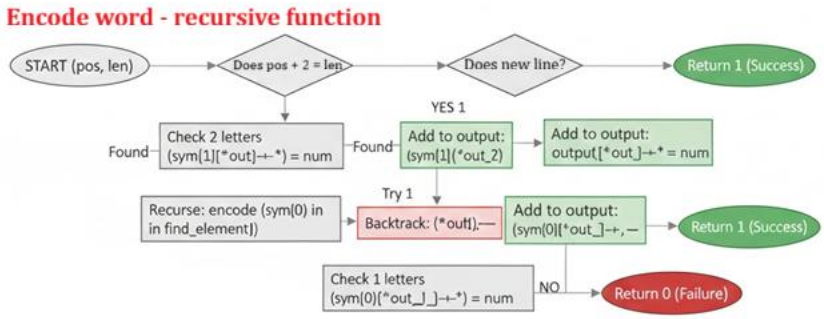
Wypisz wynik

|

STOP

Schemat blokowy (szczegółowy – graficzny):





Użyte funkcje:

- find_element() – szuka symbolu pierwiastka w tablicy

- encode_word() – rekurencyjnie szyfruje słowo

- strtok() – dzieli tekst na słowa

- tolower() – normalizuje wielkość liter

5. Testy poprawności działania

PRZYKŁADY:

Wejście: SobOta rANo

Wyjście: 16*8*5*8*73**88*102

```
### SZYFROWANIE SYMBOLAMI PIERWIASTKOW ###
1. Rozpocznij szyfrowanie
2. Wyjdz z programu
Wybierz opcje (1 lub 2): 1

--- Rozpoczeto Szyfrowanie ---
Podaj tekst do zaszyfrowania: SobOta rANo
16*8*5*8*73**88*102
```

Wejście: He Na

Wyjście: 2**11

```
### SZYFROWANIE SYMBOLAMI PIERWIASTKOW ###
1. Rozpocznij szyfrowanie
2. Wyjdz z programu
Wybierz opcje (1 lub 2): 1

--- Rozpoczeto Szyfrowanie ---
Podaj tekst do zaszyfrowania: he na
2**11
```

Przypadki błędne:

Wejście: kot

Wyjście: Nie da się zaszyfrować...

```
### SZYFROWANIE SYMBOLAMI PIERWIASTKOW ###
1. Rozpocznij szyfrowanie
2. Wyjdz z programu
Wybierz opcje (1 lub 2): 1

--- Rozpoczeto Szyfrowanie ---
Podaj tekst do zaszyfrowania: kot

Nie da sie zaszyfrowac podanego tekstu tym sposobem.
```

6. Wnioski

Projekt pozwolił zrozumieć działanie rekurencji, backtrackingu oraz pracę z tablicami struktur. Nauczyłem się też obsługi wejścia znakowego i analizowania danych pod kątem wyjątków w algorytmie.